In [23]:
```
import seaborn as sns
print(sns.get_dataset_names())
data = sns.load_dataset('flights')
```

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes', 'diamonds',
'dots', 'dowjones', 'exercise', 'flights', 'fmri', 'geyser', 'glue', 'healthexp',
'iris', 'mpg', 'penguins', 'planets', 'seaice', 'taxis', 'tips', 'titanic']
```

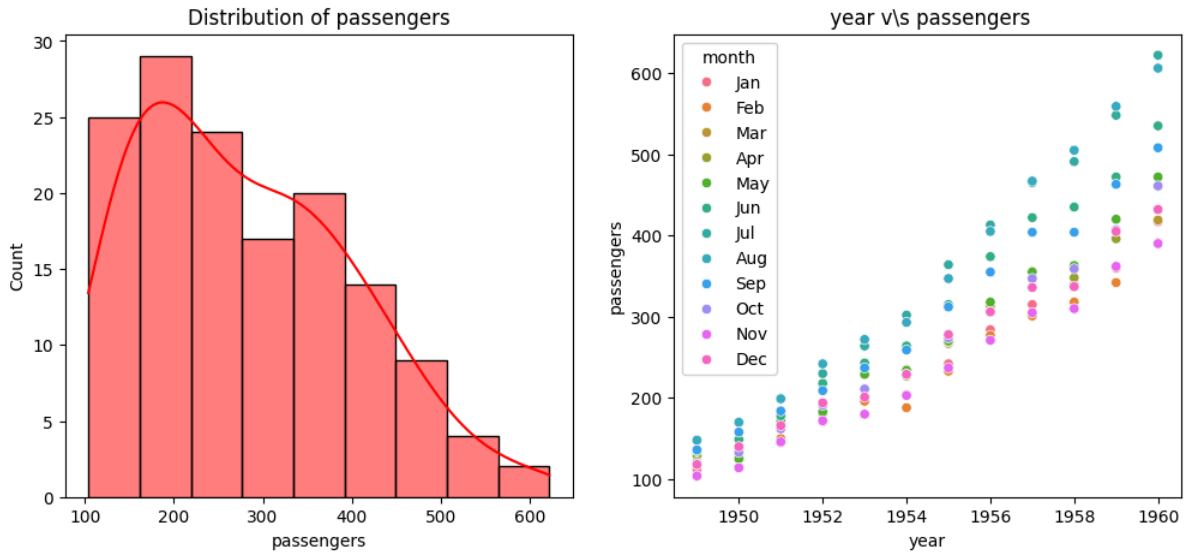
In [24]:
```
data.describe(include='all')
```

Out[24]:

	year	month	passengers
count	144.000000	144	144.000000
unique	NaN	12	NaN
top	NaN	Jan	NaN
freq	NaN	12	NaN
mean	1954.500000	NaN	280.298611
std	3.464102	NaN	119.966317
min	1949.000000	NaN	104.000000
25%	1951.750000	NaN	180.000000
50%	1954.500000	NaN	265.500000
75%	1957.250000	NaN	360.500000
max	1960.000000	NaN	622.000000

In [25]:
```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2, figsize = (12, 5))
sns.histplot(data['passengers'], kde=True, ax = axes[0], color = 'red')
axes[0].set_title("Distribution of passengers")
sns.scatterplot(x='year', y='passengers', data=data, hue="month", ax=axes[1])
axes[1].set_title(" year v\s passengers")
plt.show()
```

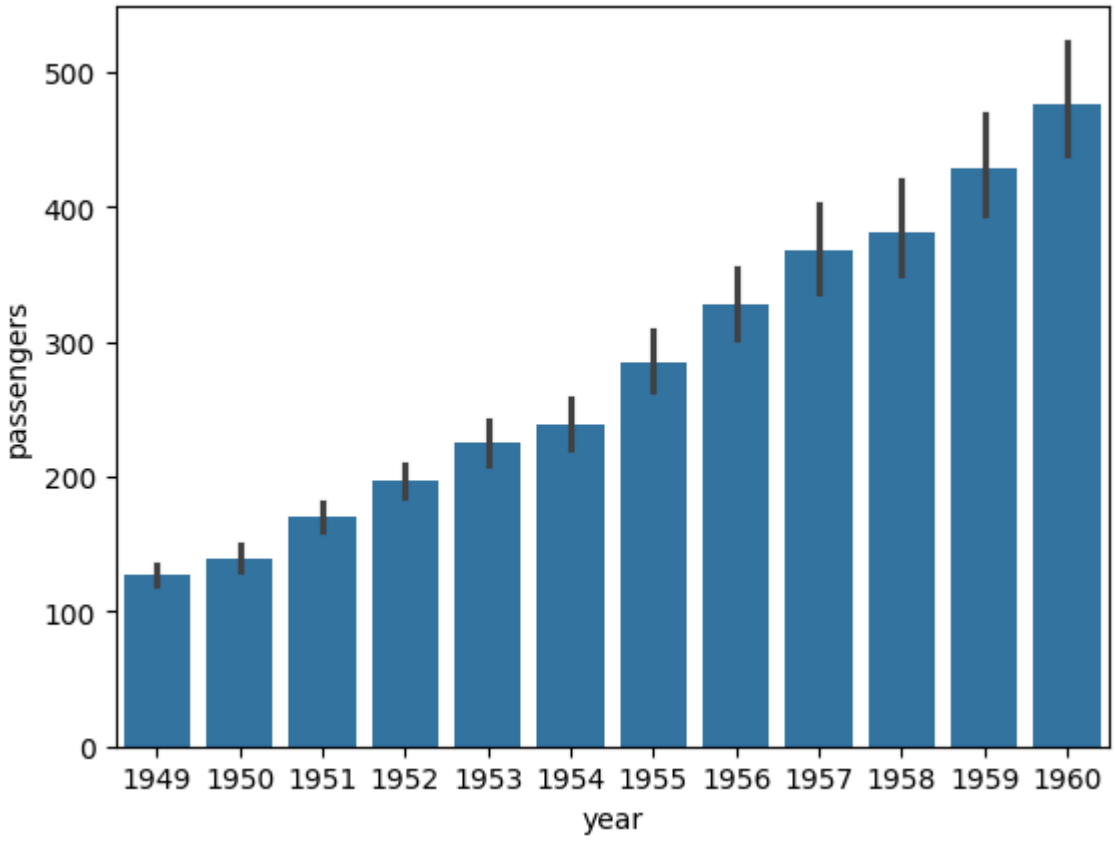
```
<>:6: SyntaxWarning: invalid escape sequence '\s'
<>:6: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_242/583724054.py:6: SyntaxWarning: invalid escape sequence '\s'
  axes[1].set_title(" year v\s passengers")
```



Task2: Flights Dataset: plot analysis

In [26]:
```
# How did the total numbers of airline passengers change from 1949 to 1960
sns.barplot(data=data, x='year', y='passengers')
```

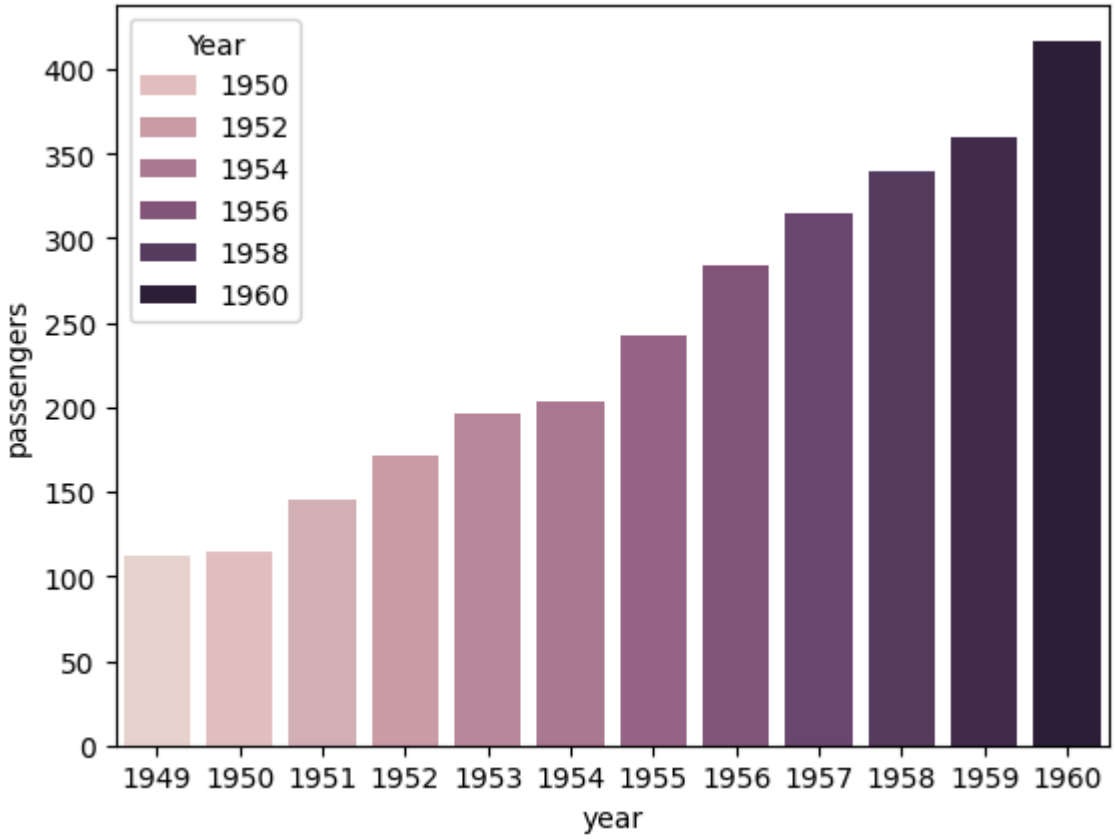
Out[26]:
```
<Axes: xlabel='year', ylabel='passengers'>
```



How did passenger count change over years for a single month(e.g, January)

In [27]:
```
jan = data[data['month'] == 'Jan']
sns.barplot(data=jan, x='year', y='passengers', hue='year')
plt.legend(title = "Year")
```

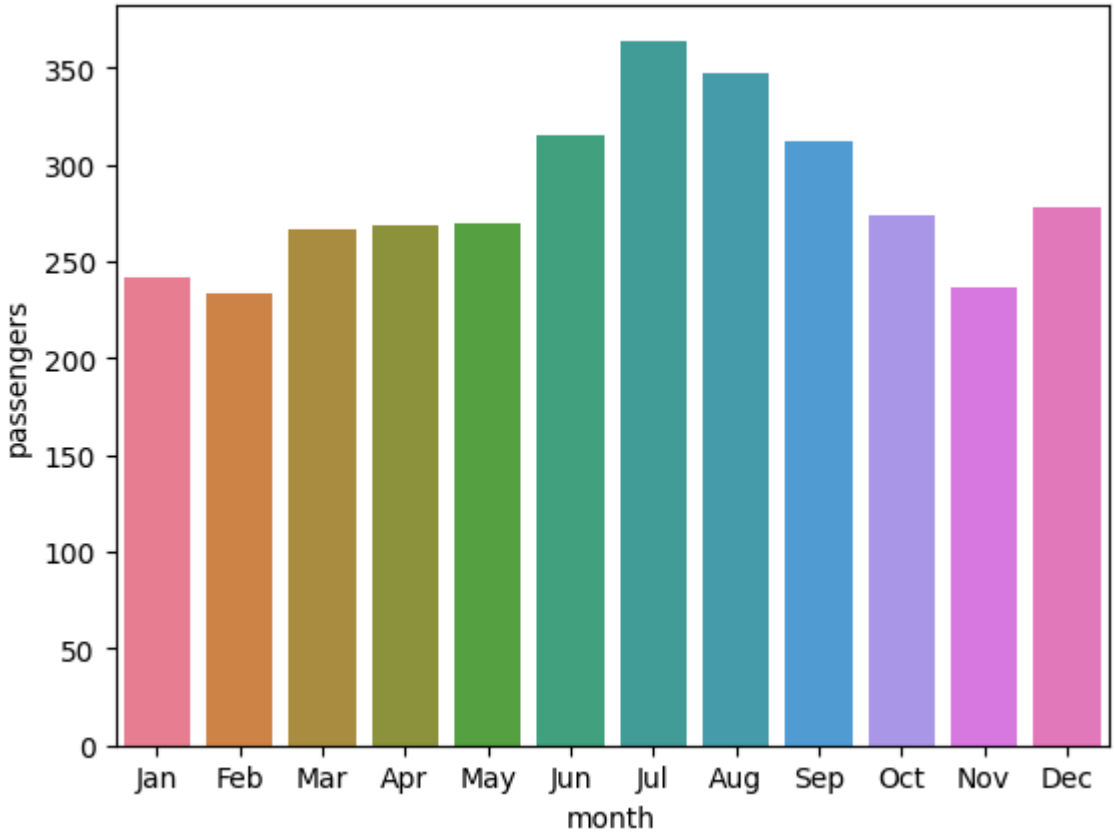
Out[27]:
```
<matplotlib.legend.Legend at 0x7e1edddb5280>
```



In a chosen years(e.g, 1955), which months had higher passengers counts?

In [28]:
```
data_1955 = data[data['year'] == 1955]
sns.barplot(data=data_1955, x='month', y='passengers', hue='month')
```

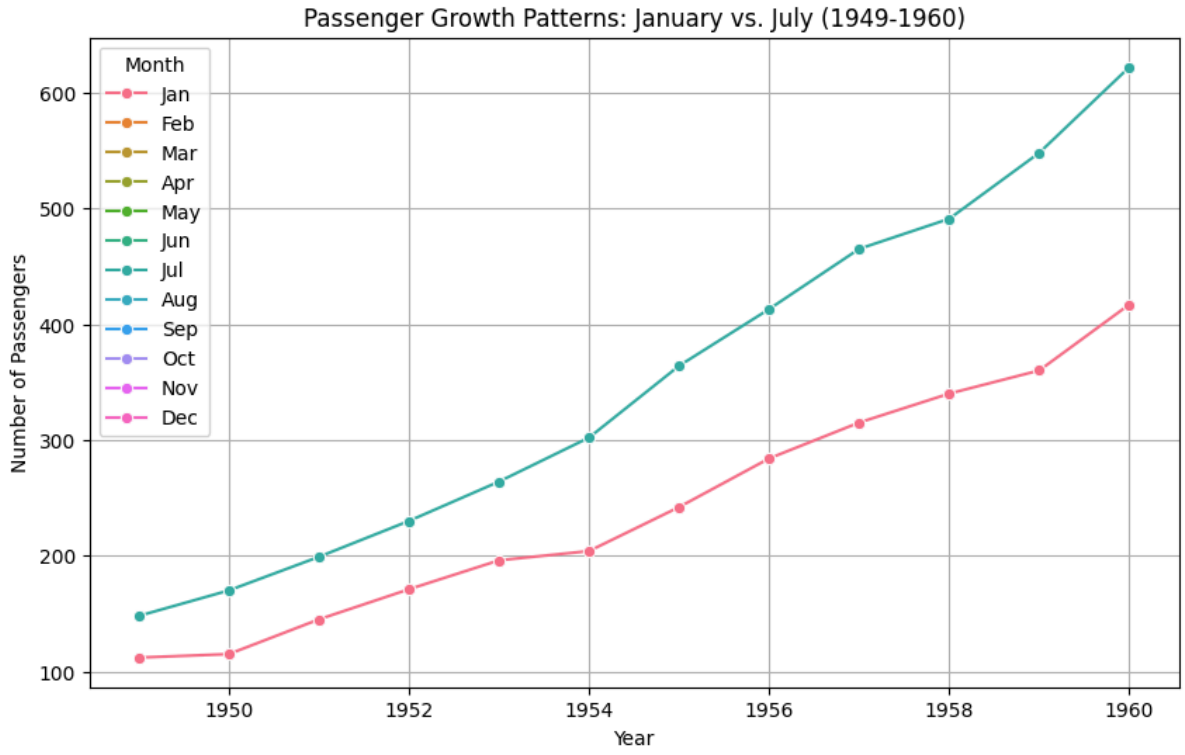
Out[28]:
```
<Axes: xlabel='month', ylabel='passengers'>
```



From 1945 - 1960 does january and july show similar growth patterns over the year comparison.

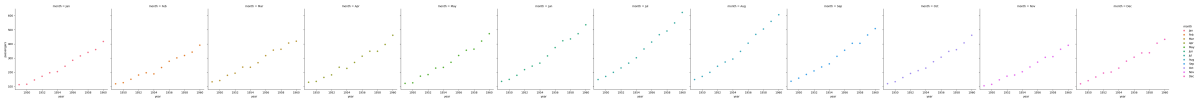
In [29]:
```
jan_jul_data = data[data['month'].isin(['Jan', 'Jul'])]

plt.figure(figsize=(10, 6))
sns.lineplot(data=jan_jul_data, x='year', y='passengers', hue='month', marker='o')
plt.title('Passenger Growth Patterns: January vs. July (1949-1960)')
plt.xlabel('Year')
plt.ylabel('Number of Passengers')
plt.legend(title='Month')
plt.grid(True)
plt.show()
```



In [30]:
```
sns.relplot(data=data, x='year', y='passengers', col='month', hue='month', kind='sc
```

Out[30]:
```
<seaborn.axisgrid.FacetGrid at 0x7e1f1c25b1a0>
```



In [31]:
```
# Check for missing values
titanic_data = sns.load_dataset('titanic')
titanic_data.isnull().any()
```

Out[31]:

	0
survived	False
pclass	False
sex	False
age	True
sibsp	False
parch	False
fare	False
embarked	True
class	False
who	False
adult_male	False
deck	True
embark_town	True
alive	False
alone	False

dtype: bool

In [32]:

```
# Drop missing values
drop_data = titanic_data.dropna(inplace=False)
print(drop_data)
```

```
     survived  pclass     sex   age  sibsp  parch     fare embarked  class  \
1           1       1  female  38.0      1      0  71.2833        C  First
3           1       1  female  35.0      1      0  53.1000        S  First
6           0       1    male  54.0      0      0  51.8625        S  First
10          1       3  female   4.0      1      1  16.7000        S  Third
11          1       1  female  58.0      0      0  26.5500        S  First
..        ...     ...     ...   ...    ...    ...      ...      ...    ...
871         1       1  female  47.0      1      1  52.5542        S  First
872         0       1    male  33.0      0      0   5.0000        S  First
879         1       1  female  56.0      0      1  83.1583        C  First
887         1       1  female  19.0      0      0  30.0000        S  First
889         1       1    male  26.0      0      0  30.0000        C  First

       who  adult_male deck  embark_town alive  alone
1    woman       False    C    Cherbourg   yes  False
3    woman       False    C  Southampton   yes  False
6      man        True    E  Southampton    no   True
10   child       False    G  Southampton   yes  False
11   woman       False    C  Southampton   yes   True
..     ...         ...  ...          ...   ...    ...
871  woman       False    D  Southampton   yes  False
872    man        True    B  Southampton    no   True
879  woman       False    C    Cherbourg   yes  False
887  woman       False    B  Southampton   yes   True
889    man        True    C    Cherbourg   yes   True

[182 rows x 15 columns]
```

In [33]:
```
# Fill missing values
data_subset = titanic_data
data_subset['age'].fillna(data_subset['age'].mean())
```

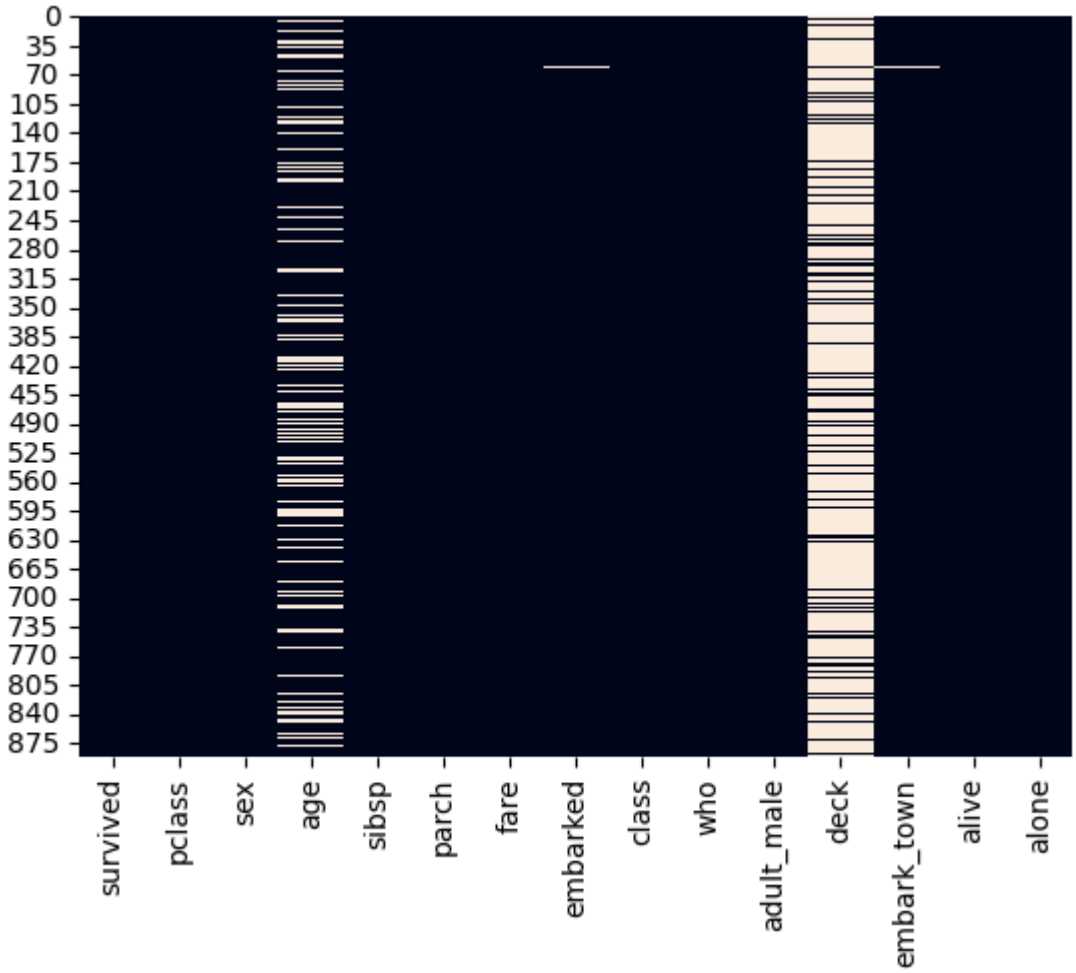
Out[33]:

	age
0	22.000000
1	38.000000
2	26.000000
3	35.000000
4	35.000000
...	...
886	27.000000
887	19.000000
888	29.699118
889	26.000000
890	32.000000

891 rows × 1 columns

dtype: float64

In [34]:
```
# Vizualize the missing data
sns.heatmap(titanic_data.isnull(), cbar=False)
plt.show()
```

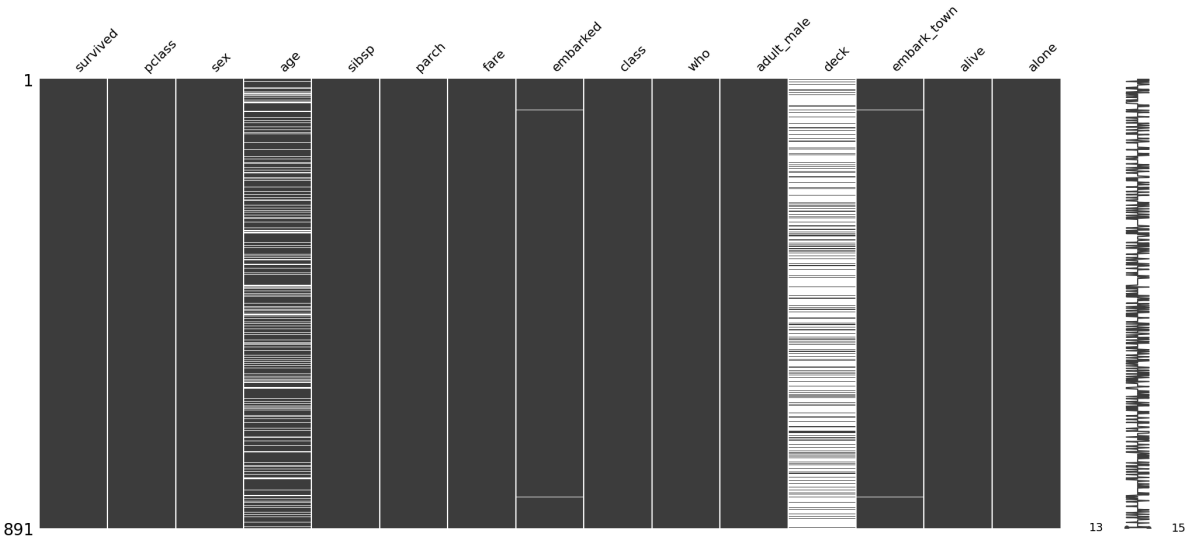


In [35]:
```
!pip install missingno
import missingno as msno
```

```
Requirement already satisfied: missingno in /usr/local/lib/python3.12/dist-package
s (0.5.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (f
rom missingno) (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packag
es (from missingno) (3.10.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (f
rom missingno) (1.16.3)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages
(from missingno) (0.13.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-
packages (from matplotlib->missingno) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-pack
ages (from matplotlib->missingno) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist
-packages (from matplotlib->missingno) (4.62.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist
-packages (from matplotlib->missingno) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-p
ackages (from matplotlib->missingno) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-package
s (from matplotlib->missingno) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-
packages (from matplotlib->missingno) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib->missingno) (2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packa
ges (from seaborn->missingno) (2.2.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-pack
ages (from pandas>=1.2->seaborn->missingno) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-pa
ckages (from pandas>=1.2->seaborn->missingno) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
(from python-dateutil>=2.7->matplotlib->missingno) (1.17.0)
```

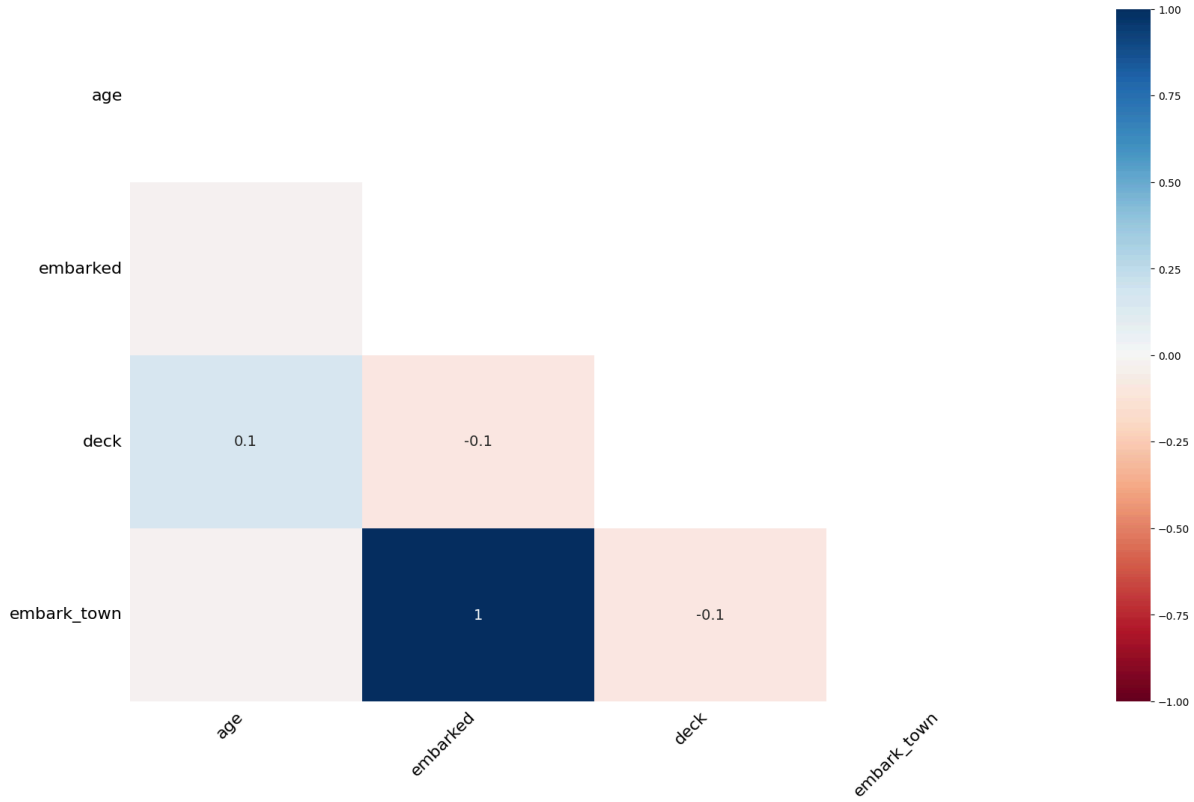
In [36]:
```
msno.matrix(titanic_data)
```

Out[36]:
```
<Axes: >
```



In [37]:
```
msno.heatmap(titanic_data)
```

Out[37]:
```
<Axes: >
```



In [38]:
```
planets_data = sns.load_dataset('planets')
planets_data.head()
```

Out[38]:

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

In [39]:
```
planets_data.describe(include='all')
```

Out[39]:

	method	number	orbital_period	mass	distance	year
count	1035	1035.000000	992.000000	513.000000	808.000000	1035.000000
unique	10	NaN	NaN	NaN	NaN	NaN
top	Radial Velocity	NaN	NaN	NaN	NaN	NaN
freq	553	NaN	NaN	NaN	NaN	NaN
mean	NaN	1.785507	2002.917596	2.638161	264.069282	2009.070531
std	NaN	1.240976	26014.728304	3.818617	733.116493	3.972567
min	NaN	1.000000	0.090706	0.003600	1.350000	1989.000000
25%	NaN	1.000000	5.442540	0.229000	32.560000	2007.000000
50%	NaN	1.000000	39.979500	1.260000	55.250000	2010.000000
75%	NaN	2.000000	526.005000	3.040000	178.500000	2012.000000
max	NaN	7.000000	730000.000000	25.000000	8500.000000	2014.000000

which columns has the highest number of missing values

In [40]:
```
planets_data.isnull().sum().sort_values(ascending=False)
```

Out[40]:

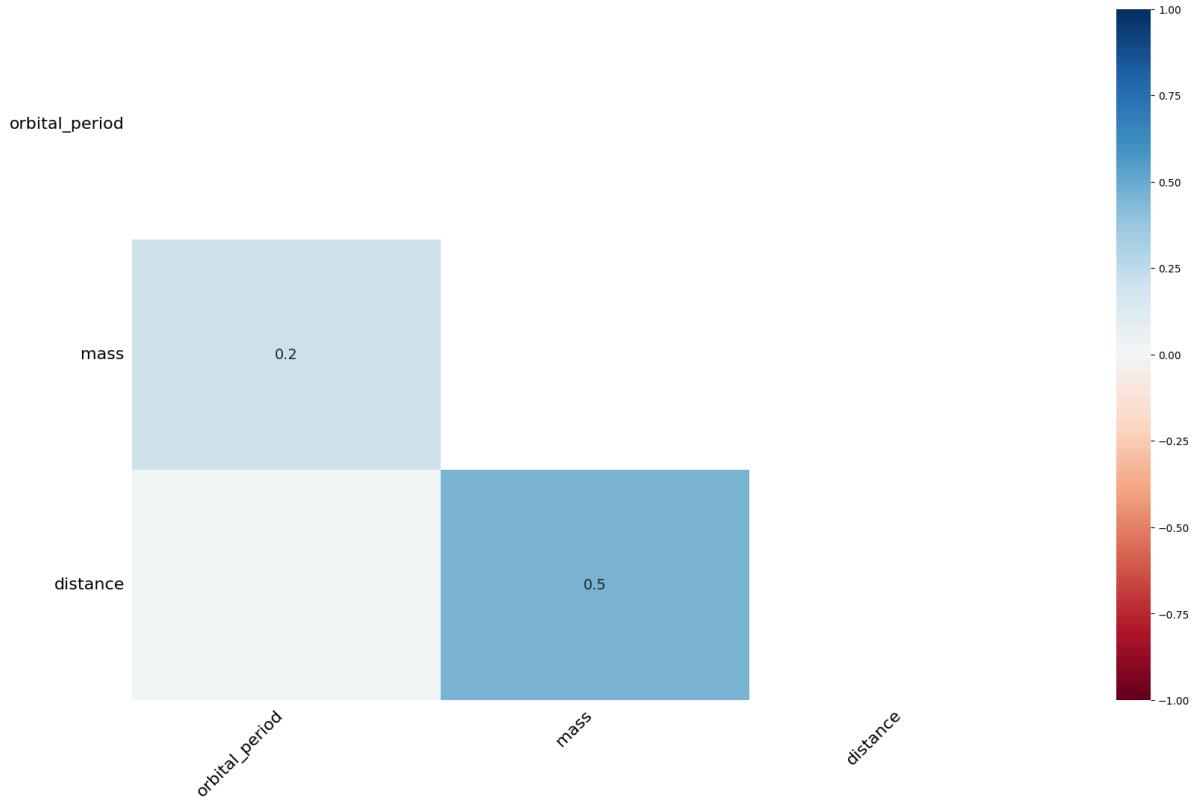
	0
mass	522
distance	227
orbital_period	43
method	0
number	0
year	0

dtype: int64

Which two columns have the strongest missing-value relationship

In [41]:
```
msno.heatmap(planets_data)
```

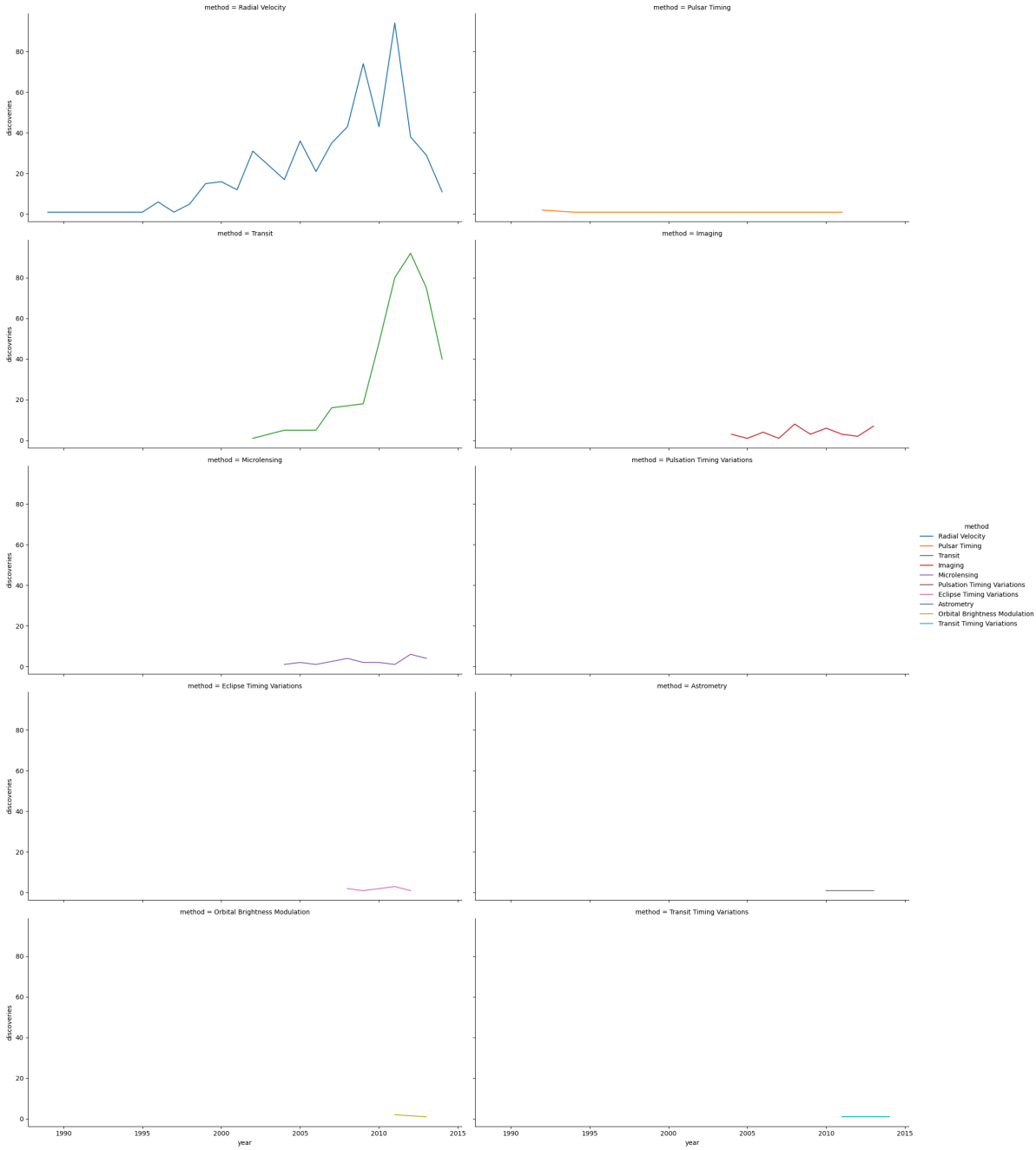
Out[41]:
```
<Axes: >
```



Which method shows a sharp rise in discoveries after 2009

In [42]:
```
disc = planets_data.groupby(['year', 'method']).size().reset_index(name='discoverie
sns.relplot(
    data=disc,
    x='year',
    y='discoveries',
    hue='method',
    col='method',
    kind='line',
    aspect=2,
    col_wrap=2
)
```

Out[42]:
```
<seaborn.axisgrid.FacetGrid at 0x7e1edcba35f0>
```



Fill the missing values of the distacne columns using column mean.

In [43]:
```
planets_data.fillna(planets_data['distance'].mean())
```

Out[43]:

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300000	7.100000	77.40	2006
1	Radial Velocity	1	874.774000	2.210000	56.95	2008
2	Radial Velocity	1	763.000000	2.600000	19.84	2011
3	Radial Velocity	1	326.030000	19.400000	110.62	2007
4	Radial Velocity	1	516.220000	10.500000	119.47	2009
...	...	...	...	...	...	...
1030	Transit	1	3.941507	264.069282	172.00	2006
1031	Transit	1	2.615864	264.069282	148.00	2007
1032	Transit	1	3.191524	264.069282	174.00	2007
1033	Transit	1	4.125083	264.069282	293.00	2008
1034	Transit	1	4.187757	264.069282	260.00	2008

1035 rows × 6 columns

Task 4. Plotting with seaborn

Vizualize the relatonship between the total bill amount and tip amount differentiate smokers and non-smokers and represent gender.

In [44]:

```
import seaborn as sns
tip_data = sns.load_dataset('tips')
tip_data.head()
```

Out[44]:

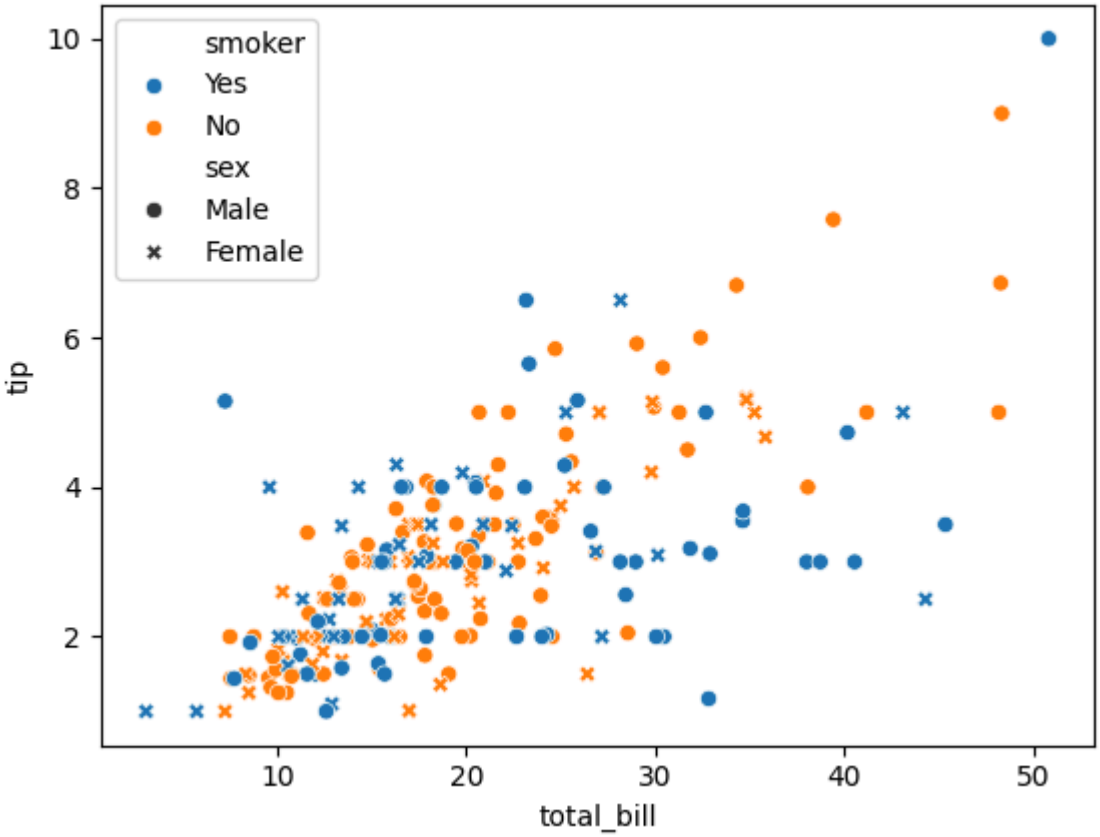
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

In [45]:

```
sns.scatterplot(data = tip_data, x='total_bill', y='tip', hue='smoker', style='sex'
```

Out[45]:

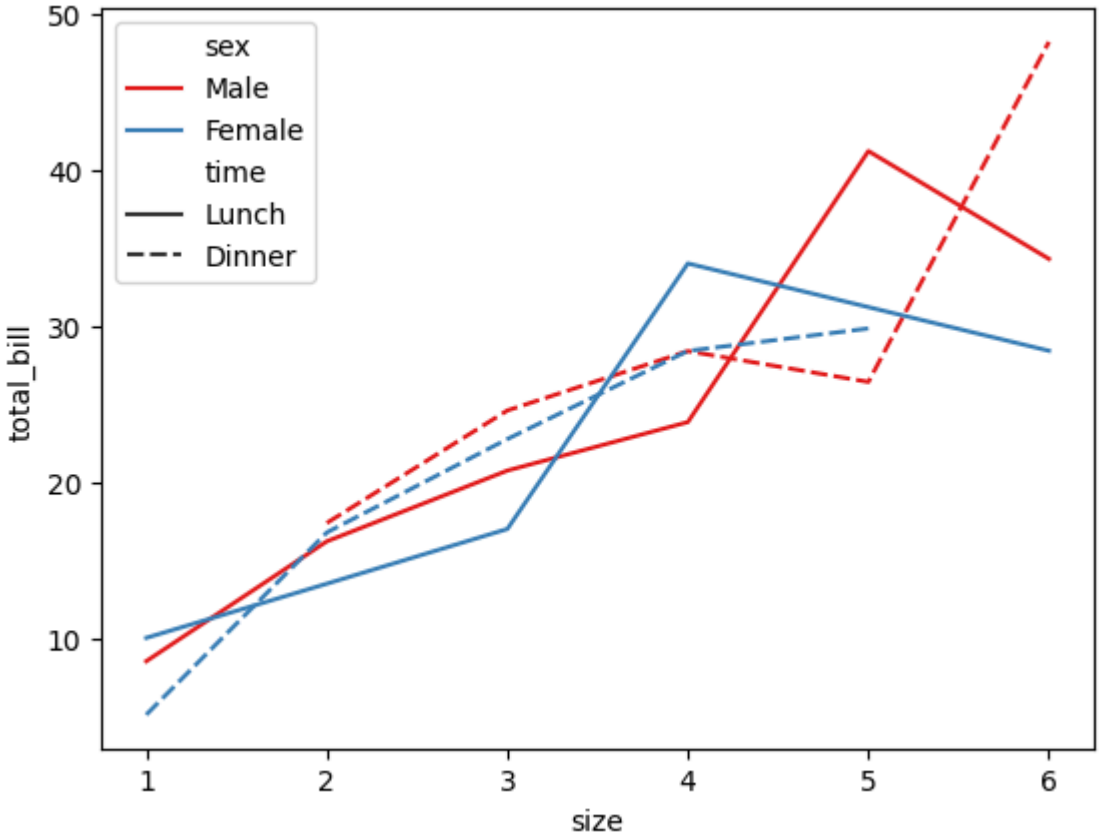
```
<Axes: xlabel='total_bill', ylabel='tip'>
```



Show how the average total bill changes with table size. Compare males and females separately and also distinguish lunch and dinner.

In [46]:
```
sns.lineplot(data=tip_data, x='size', y='total_bill', hue='sex', style='time', esti
```

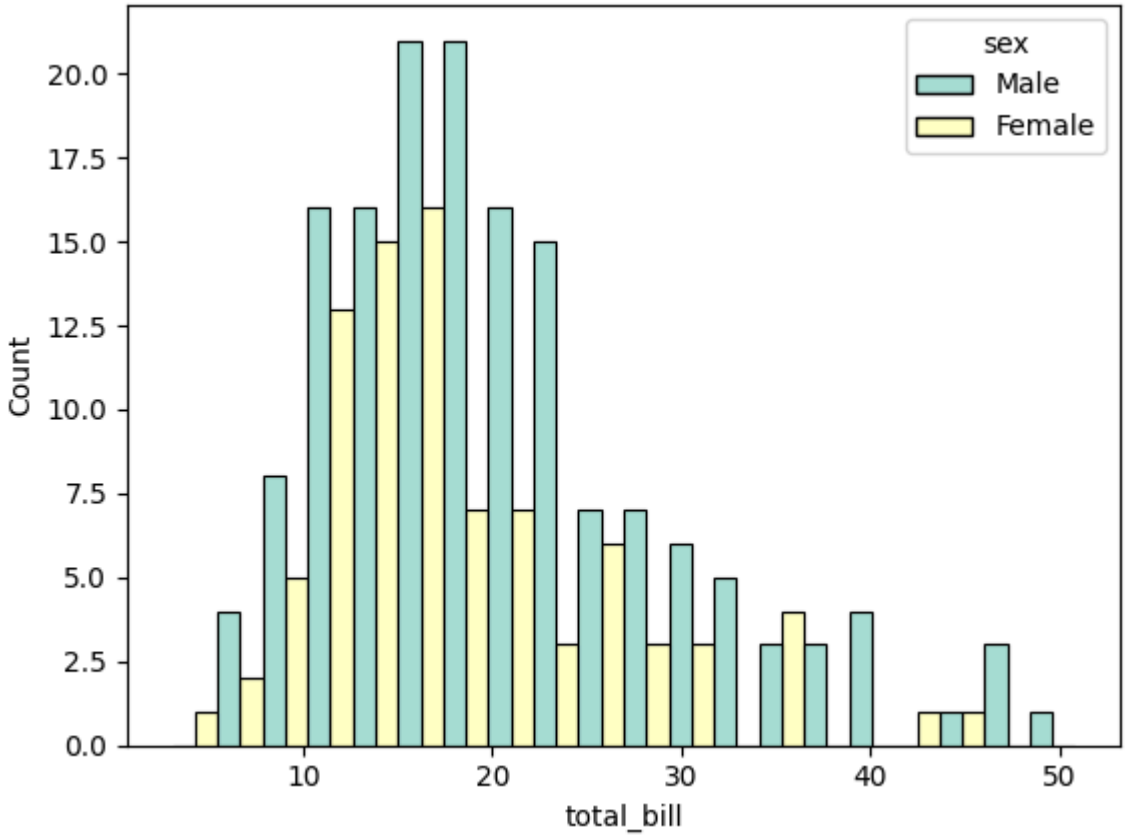
Out[46]:
```
<Axes: xlabel='size', ylabel='total_bill'>
```



PLot the distribution of total bills amounts for males and females together. Use density instead of frequnecy and include smooth distribution

In [47]:
```
sns.histplot(data=tip_data, bins = 20, x='total_bill', palette='Set3', hue='sex', m
```

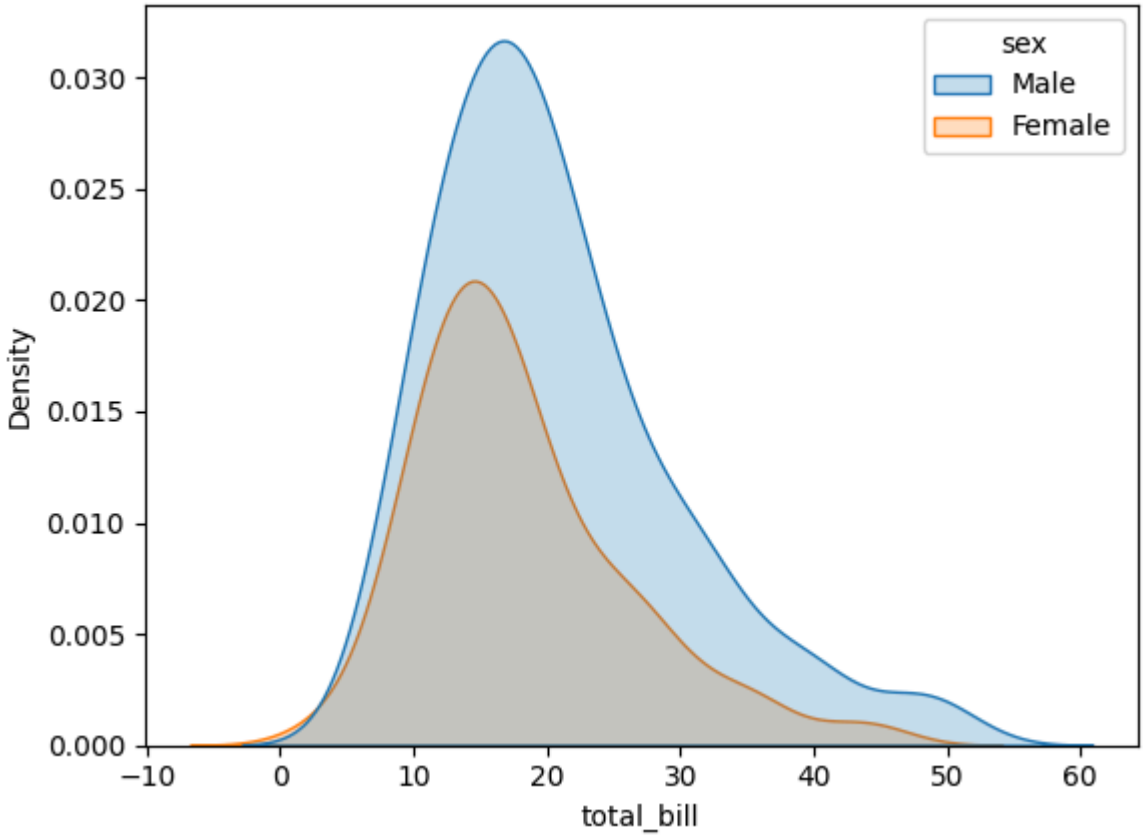
Out[47]:
```
<Axes: xlabel='total_bill', ylabel='Count'>
```



Compare the probability density distribution of total bills for males adn females.

In [48]:
```
sns.kdeplot(data=tip_data, x='total_bill', hue='sex', fill=True)
```

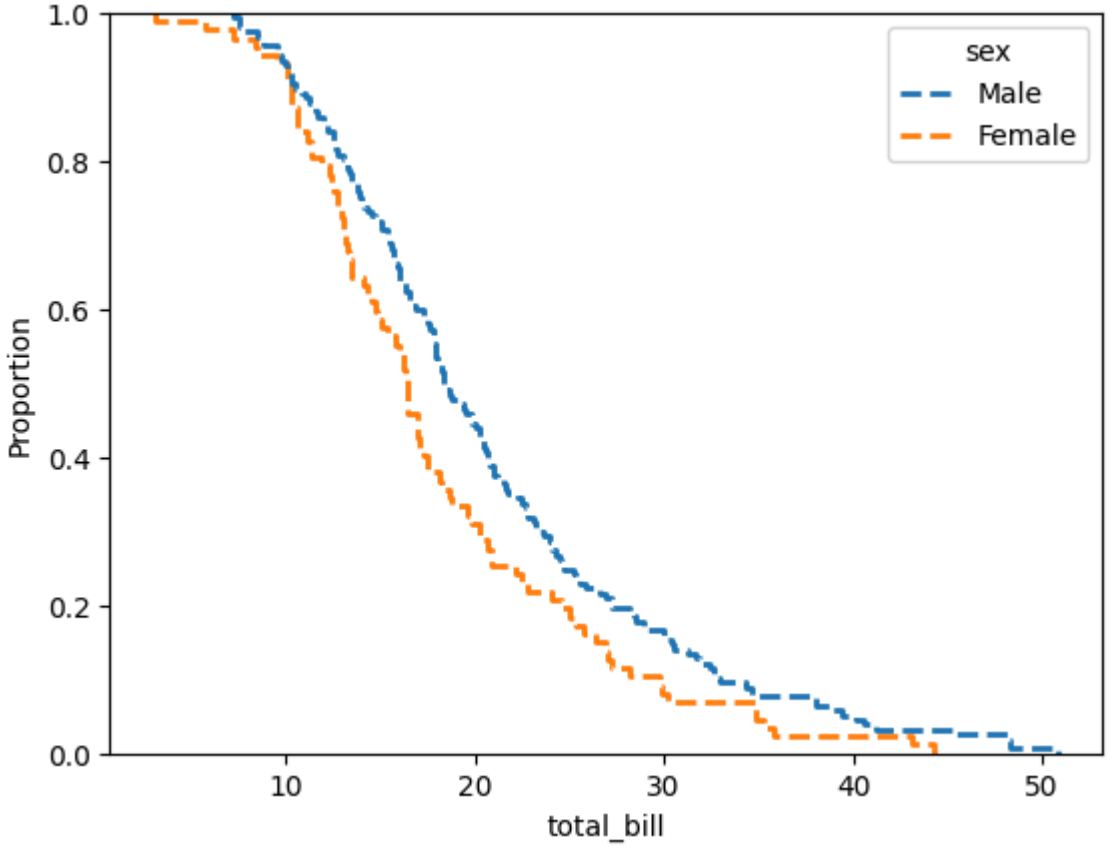
Out[48]:
```
<Axes: xlabel='total_bill', ylabel='Density'>
```



Plot the cumulative probability of total bill amounts for males and female(Use dashed curve)

In [49]:
```
sns.ecdfplot(data=tip_data, x='total_bill', hue='sex', linestyle='--', linewidth=2,
```

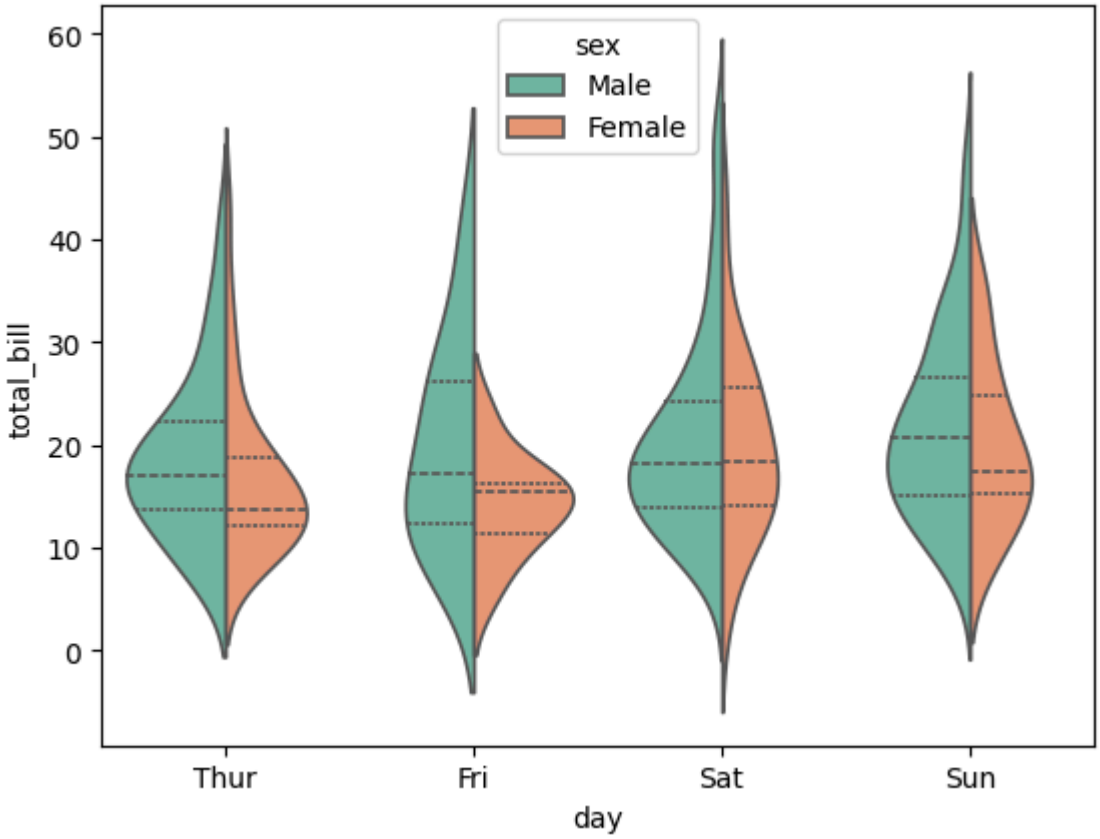
Out[49]:
```
<Axes: xlabel='total_bill', ylabel='Proportion'>
```



Show the variation in spending behaviour for males and females across different days of the week

In [50]:
```
sns.violinplot(data=tip_data, x='day', y='total_bill', hue='sex', bw_method='scott'
```

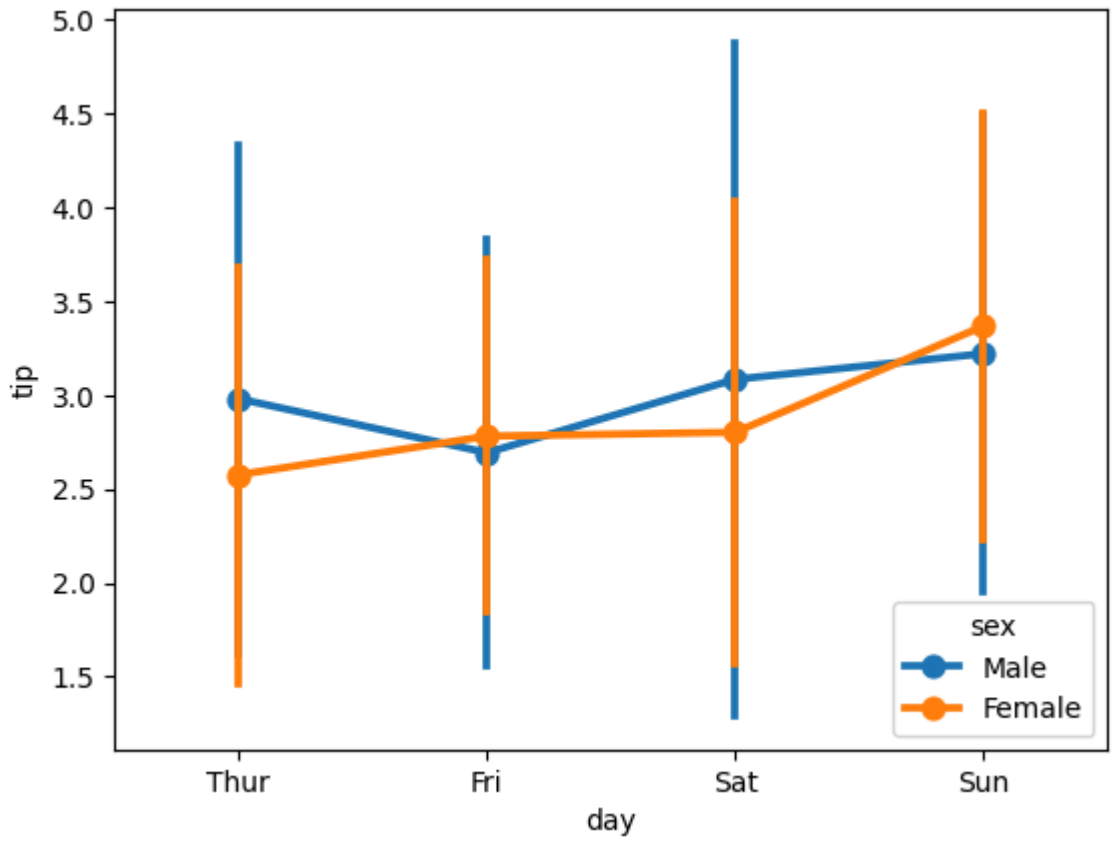
Out[50]:
```
<Axes: xlabel='day', ylabel='total_bill'>
```



Plot the average tip amount for each day of the week and compare for males and females. Show the variability of the estimate.

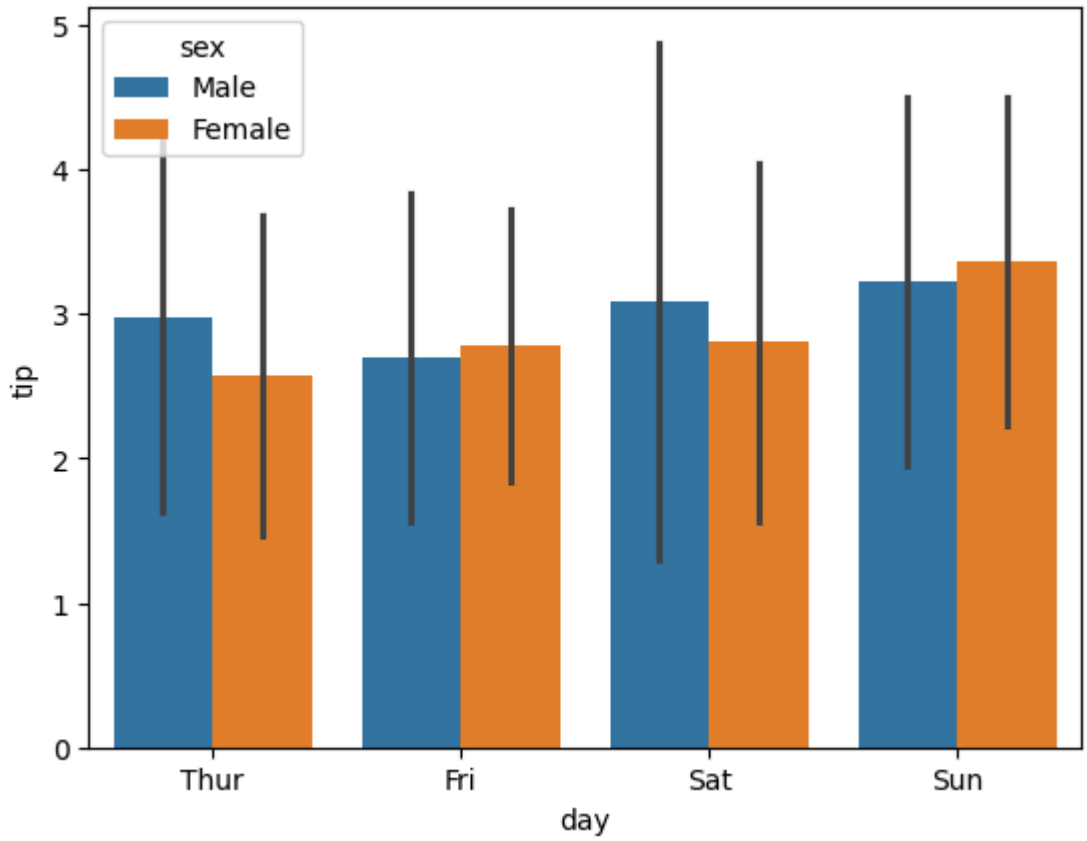
In [51]:
```
sns.pointplot(data=tip_data, x='day', y='tip', hue='sex', errorbar='sd')
```

Out[51]:
```
<Axes: xlabel='day', ylabel='tip'>
```



In [52]:
```
sns.barplot(data=tip_data, x='day', y='tip', hue='sex', errorbar='sd', estimator="m
```

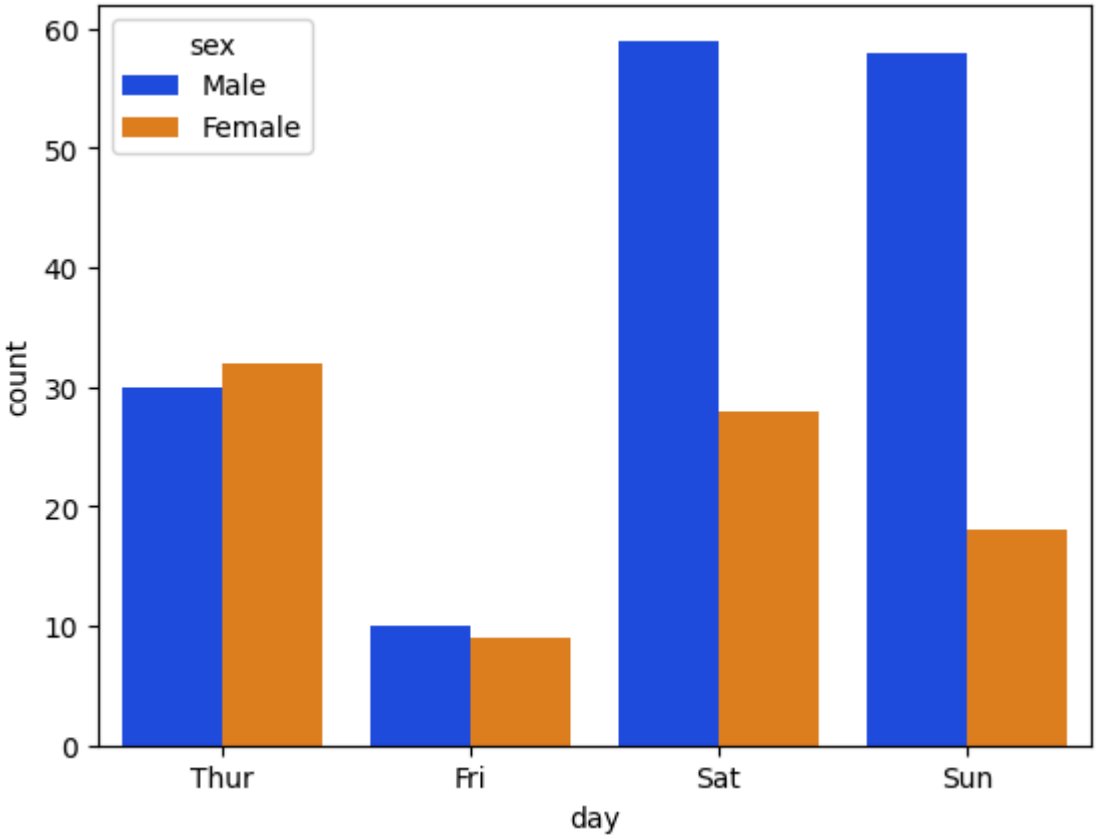
Out[52]:
```
<Axes: xlabel='day', ylabel='tip'>
```



Display the number of the customers visiting on each day of the week and compare counts between males and females.

In [53]:
```
sns.countplot(data=tip_data, x='day', hue='sex', palette='bright')
```

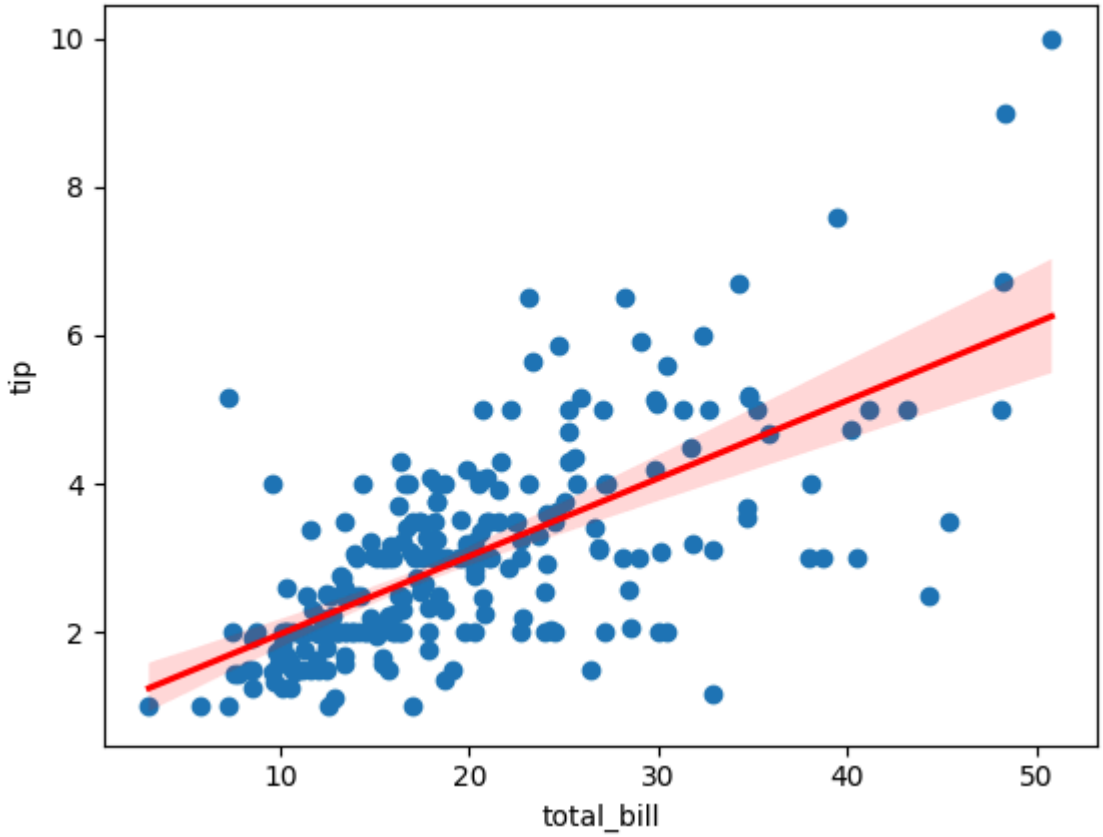
Out[53]:
```
<Axes: xlabel='day', ylabel='count'>
```



Showing how tip amount depends on total bill and display confidence intervals for the prediciton line.

In [54]:
```
sns.regplot(data=tip_data, x='total_bill', y='tip', ci=95, scatter_kws={'alpha': 1}
```

Out[54]:
```
<Axes: xlabel='total_bill', ylabel='tip'>
```



In [54]: